

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FCCD – Software



Tarea:

Práctica Experimental Unidad I

Materia:

Aplicaciones Web

Integrantes:

Álava Alvarado Jean Pierre
Rivera Suárez Jonathan Javier
Tejada Bajaan Luis Alejandro

Docente:

Ing. Guerrero Ulloa Gleiston Ciceron

Curso:

5to Software “A”

Quevedo – Los Ríos – Ecuador

10/06/2026

Índice

1. Introducción	1
2. Fundamento teórico – Unidad I	1
2.1. Taxonomía de Aplicaciones Web y Ciclo HTTP/HTTPS	1
2.1.1. Tipos de aplicaciones web	1
2.1.2. Ciclo de vida HTTP/HTTPS	2
2.1.3. Principios POUR de las WCAG 2.1	2
2.2. HTML5 Semántico, WAI-ARIA y Web Storage	3
2.2.1. Semántica estructural y SEO	3
2.2.2. WAI-ARIA en profundidad	3
2.2.3. Inputs HTML5 y validación nativa vs personalizada	3
2.2.4. LocalStorage vs SessionStorage vs Cookies	4
2.3. CSS3 Avanzado: Grid, Flexbox, Mobile-First y Contraste	4
2.3.1. CSS Grid vs Flexbox	4
2.3.2. Estrategia Mobile-First y tipografía fluida	4
2.3.3. Variables CSS y herencia	5
2.3.4. Fórmulas de luminancia y contraste	5
2.4. JavaScript ES6+: Event Loop, Módulos y Accesibilidad Dinámica	5
2.4.1. Arquitectura del Event Loop	5
2.4.2. Módulos ES6 vs CommonJS vs IIFE	5
2.4.3. Fetch nativo vs Axios	6
2.4.4. Accesibilidad dinámica: focus trap y aria-live	6
3. Desarrollo	6
3.1. Descripción del proyecto	6
3.2. Entorno de desarrollo	7
3.3. Estructura del proyecto	8
3.4. HTML5 semántico aplicado	8
3.5. CSS3 responsivo con Grid y Flexbox	9
3.6. JavaScript ES6+ modular	10
3.7. Consumo de API pública	10
3.8. Funcionalidades del sistema	11
3.9. Validaciones y accesibilidad	12
3.10. Validación con Lighthouse y WAVE	12
3.11. Validación W3C HTML y CSS	14
4. Conclusiones	16
5. Repositorio en GitHub	17
6. Referencias	17

1. Introducción

El presente informe técnico describe el desarrollo de la aplicación web denominada **SGROAS**, un Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad orientado a cooperativas de transporte terrestre del Ecuador. La aplicación permite visualizar un directorio de cooperativas, consultar información operativa de la flota, registrar nuevas cooperativas mediante un formulario validado y mostrar estadísticas generales del sistema.

El proyecto fue desarrollado aplicando **HTML5 semántico**, **CSS3 responsivo** con Grid y Flexbox, y **JavaScript ES6+** mediante una estructura modular. HTML5 permite organizar el contenido de una página web con etiquetas que describen mejor el significado de cada sección, favoreciendo la accesibilidad, la optimización para buscadores y la comprensión del documento por parte del navegador [1]. Por su parte, CSS Grid y Flexbox facilitan la creación de interfaces adaptables a diferentes tamaños de pantalla, lo cual es fundamental en aplicaciones modernas que deben funcionar correctamente en computadoras, tablets y dispositivos móviles [2], [3].

Además, la aplicación implementa el consumo de una API pública mediante la función `fetch()` y el uso de `async/await`, permitiendo obtener datos externos y mostrarlos dinámicamente en la interfaz. Para este proyecto se utilizó JSONPlaceholder como API pública de demostración, adaptando los datos de usuarios al contexto de cooperativas de transporte [4], [8]. Finalmente, se realizaron pruebas de calidad con Lighthouse y WAVE para analizar aspectos relacionados con rendimiento, accesibilidad, buenas prácticas, SEO y contraste visual [9], [10].

2. Fundamento teórico – Unidad I

2.1. Taxonomía de Aplicaciones Web y Ciclo HTTP/HTTPS

2.1.1. Tipos de aplicaciones web

Las aplicaciones web se clasifican según su arquitectura y comportamiento en el cliente y el servidor. Las aplicaciones estáticas sirven contenido fijo directamente desde el servidor sin procesamiento dinámico; son rápidas y de fácil despliegue, pero limitadas en interactividad. Las aplicaciones dinámicas generan contenido en tiempo de ejecución mediante lenguajes de servidor como PHP, Node.js o Python, permitiendo personalización y acceso a bases de datos [14].

Las Single Page Applications (SPA) representan un paradigma moderno donde toda la lógica de renderizado se delega al navegador. Frameworks como React, Vue y Angular cargan un único documento HTML y actualizan el DOM de forma parcial mediante JavaScript, logrando experiencias de usuario fluidas sin recarga de página completa. Esta arquitectura mejora la percepción de velocidad pero presenta desafíos en SEO si no se implementa correctamente el renderizado del lado del servidor [15].

Las Progressive Web Apps (PWA) añaden a las SPA capacidades nativas como funcionamiento offline mediante Service Workers, instalación en el dispositivo y notificaciones push. A diferencia de una SPA pura, la PWA incorpora un archivo `manifest.json` y estrategias de caché que la hacen funcionar incluso sin conexión a internet. Las arquitecturas monolíticas concentran toda la lógica en una sola aplicación, mientras que los microservicios distribuyen las funcionalidades en servicios independientes que se comunican mediante APIs [20].

Criterio	SPA	PWA
Funcionamiento offline	No	Sí, Service Worker
Instalable en dispositivo	No	Sí, <code>manifest.json</code>
Notificaciones push	No	Sí
SEO nativo	Limitado	Limitado
Velocidad percibida	Alta	Muy alta
Complejidad de implementación	Media	Alta

Cuadro 1: Comparativa entre SPA y PWA

2.1.2. Ciclo de vida HTTP/HTTPS

Toda comunicación entre cliente y servidor sigue el protocolo HTTP o su versión segura HTTPS. El ciclo inicia con una resolución DNS que traduce el nombre de dominio a una dirección IP. A continuación, se establece una conexión TCP y en HTTPS se negocia el protocolo TLS mediante un handshake que autentica el servidor y acuerda claves de cifrado simétricas para la sesión [14].

Los métodos HTTP principales son GET para obtener recursos, POST para enviar datos, PUT para actualizar un recurso completo, DELETE para eliminarlo y PATCH para actualizaciones parciales. Los códigos de estado clasifican las respuestas: 2xx indica éxito, 3xx redirección, 4xx error del cliente como 404 Not Found, y 5xx error del servidor. La cabecera HSTS, HTTP Strict Transport Security, fuerza conexiones seguras en visitas futuras, mientras que la Content Security Policy, CSP, mitiga ataques XSS declarando las fuentes de contenido permitidas [14].

2.1.3. Principios POUR de las WCAG 2.1

Las Web Content Accessibility Guidelines 2.1 organizan los criterios de accesibilidad bajo cuatro principios conocidos como POUR. Perceptible exige que la información sea presentada en formas que el usuario pueda percibir, incluyendo alternativas de texto para contenido no textual. Operable requiere que toda la funcionalidad sea accesible mediante teclado sin requerir acciones que no puedan realizarse con teclado. Comprensible implica que el contenido sea legible, predecible y que los formularios tengan asistencia para evitar errores. Robusto demanda compatibilidad con tecnologías de asistencia actuales y futuras mediante el uso correcto de HTML semántico y ARIA [16].

En Ecuador, la Ley Orgánica de Discapacidades (LOD) y la Norma Técnica INEN 2620 establecen obligaciones legales para que los servicios digitales públicos sean accesibles. El incumplimiento puede derivar en sanciones administrativas y en la exclusión de personas con discapacidad del acceso a servicios esenciales [16].

Herramienta	Tipo	Ventaja principal
Lighthouse	Automatizada integrada en Chrome	Mide Performance, Accessibility, Best Practices y SEO con puntuación 0–100
WAVE	Visual en el navegador	Muestra errores directamente sobre la página, ideal para revisión manual
AXE	API integrable en CI/CD	Menor tasa de falsos positivos, integrable en pipelines de desarrollo

Cuadro 2: Comparativa entre WAVE, Lighthouse y AXE

2.2. HTML5 Semántico, WAI-ARIA y Web Storage

2.2.1. Semántica estructural y SEO

HTML5 introduce elementos de contenido semántico que reemplazaron el uso indiscriminado de `div` y `span`. Etiquetas como `header`, `nav`, `main`, `article`, `section`, `aside` y `footer` describen el propósito del contenido, no solo su presentación. Esta semántica beneficia tanto al SEO, porque los motores de búsqueda interpretan mejor la jerarquía del contenido, como a los lectores de pantalla, que anuncian los landmarks de navegación al usuario [17].

La especificación WHATWG establece diferencias claras entre `article`, `section` y `div`. El elemento `article` representa contenido autónomo y redistribuible como una noticia o una tarjeta de cooperativa. El elemento `section` agrupa contenido temáticamente relacionado que forma parte de un todo mayor. El elemento `div` es un contenedor genérico sin valor semántico y debe usarse únicamente cuando ningún otro elemento semántico aplica [17].

2.2.2. WAI-ARIA en profundidad

La especificación WAI-ARIA define roles, estados y propiedades que enriquecen la semántica de los elementos HTML para tecnologías de asistencia. Los roles fundamentales incluyen `navigation`, `dialog`, `alert`, `status` y `region`. Los estados dinámicos como `aria-expanded`, `aria-selected` y `aria-checked` comunican el estado actual de un componente interactivo. Las propiedades `aria-label`, `aria-labelledby` y `aria-describedby` proveen contexto textual para elementos sin contenido visible suficiente [16].

En SGROAS se implementaron estos atributos extensivamente: el botón del menú hamburger usa `aria-expanded` para indicar si el menú está abierto o cerrado, el modal usa `role="dialog"` con `aria-modal="true"`, cada campo del formulario está vinculado con su mensaje de error mediante `aria-describedby`, y las regiones de carga y alerta usan `aria-live="polite"` y `aria-live="assertive"` respectivamente para que los lectores de pantalla anuncien los cambios de contenido dinámico.

2.2.3. Inputs HTML5 y validación nativa vs personalizada

HTML5 amplió el atributo `type` del elemento `input` incorporando tipos especializados que mejoran la experiencia en dispositivos móviles al mostrar teclados virtuales adecuados. SGROAS implementa los tipos `text`, `email`, `password`, `tel`, `number`, `date`, `range`, `file` y `checkbox`. La validación nativa mediante los atributos `required`, `pattern`, `min`, `max` y `minlength` permite verificar datos sin JavaScript. La validación personalizada mediante JavaScript permite

mensajes de error específicos vinculados con `aria-describedby` para que sean anunciados por lectores de pantalla [17].

2.2.4. LocalStorage vs SessionStorage vs Cookies

Característica	localStorage	sessionStorage	Cookies
Persistencia	Indefinida	Solo la sesión del tab	Configurable
Capacidad	5–10 MB	5–10 MB	4 KB
Accesible desde JS	Sí	Sí	Solo sin HttpOnly
Viaja con peticiones HTTP	No	No	Sí
Vulnerable a XSS	Sí	Sí	Solo sin HttpOnly

Cuadro 3: Comparativa entre localStorage, sessionStorage y Cookies

En SGROAS se usa `localStorage` para persistir los registros del formulario de cooperativas dado que no se almacenan credenciales ni datos sensibles [17].

2.3. CSS3 Avanzado: Grid, Flexbox, Mobile-First y Contraste

2.3.1. CSS Grid vs Flexbox

CSS Grid y Flexbox son los dos sistemas de layout modernos de CSS3. Grid es bidimensional y permite controlar filas y columnas simultáneamente, siendo ideal para estructuras de página completas. Flexbox es unidimensional y opera en un solo eje, siendo ideal para componentes internos como barras de navegación o grupos de botones [18].

Criterio	CSS Grid	Flexbox
Dimensiones	Bidimensional, filas y columnas	Unidimensional, un eje
Caso de uso principal	Layout de página completo	Componentes internos
Control de espacios vacíos	Sí mediante <code>grid-template-areas</code>	Limitado
Alineación de elementos	Excelente en ambos ejes	Excelente en el eje principal
Soporte responsive	<code>auto-fit</code> y <code>minmax</code>	<code>flex-wrap</code>
Orden visual sin cambiar el DOM	<code>grid-column</code> y <code>grid-row</code>	Propiedad <code>order</code>
Soporte de navegadores	Mayor al 96 % global	Mayor al 97 % global
Uso en SGROAS	Directorio, flota y estadísticas	Navbar, botones y formularios

Cuadro 4: Comparativa entre CSS Grid y Flexbox

2.3.2. Estrategia Mobile-First y tipografía fluida

La filosofía Mobile-First propone diseñar primero para pantallas pequeñas y luego añadir complejidad progresivamente mediante media queries con `min-width`. Esta aproximación es preferible a Desktop-First porque las restricciones del móvil obligan a priorizar el contenido esencial, y el navegador descarga primero los estilos base más ligeros [18].

Los breakpoints de SGROAS se definieron en 320px como base móvil, 600px para grids de dos columnas, 768px para la navegación horizontal y 1024px para grids de tres y cuatro columnas. La tipografía fluida se implementó con la función `clamp()` que interpola linealmente entre un tamaño mínimo y máximo según el ancho del viewport, eliminando la necesidad de redefinir tamaños en cada breakpoint. Por ejemplo: `font-size: clamp(0.95rem, 2.2vw, 1rem)` [18].

2.3.3. Variables CSS y herencia

Las Custom Properties definidas en `:root` están disponibles en todo el árbol del documento y pueden sobrescribirse en cualquier selector, permitiendo tematización eficiente. SGROAS define más de 20 tokens organizados en categorías como `-color-brand-primary`, `-spacing-md` y `-radius-lg`. El modo oscuro nativo se implementa sobrescribiendo estas variables dentro de `@media (prefers-color-scheme: dark)` sin necesidad de JavaScript [18].

2.3.4. Fórmulas de luminancia y contraste

El ratio de contraste entre texto y fondo se calcula con las fórmulas de luminancia relativa del W3C. La luminancia L de un color sRGB se obtiene convirtiendo cada canal a espacio lineal y aplicando los coeficientes: $L = 0,2126R + 0,7152G + 0,0722B$. El ratio de contraste es $(L1 + 0,05)/(L2 + 0,05)$ donde $L1$ es la luminancia del color más claro. WCAG 2.1 AA exige ratio mínimo de 4.5:1 para texto normal y 3:1 para texto grande [16].

2.4. JavaScript ES6+: Event Loop, Módulos y Accesibilidad Dinámica

2.4.1. Arquitectura del Event Loop

JavaScript es un lenguaje de un solo hilo que delega las operaciones asíncronas a las Web APIs del navegador como `fetch`, `setTimeout` y `addEventListener`. El Event Loop coordina la ejecución: el Call Stack ejecuta funciones síncronas una a una y cuando se vacía, el Event Loop mueve tareas de la Callback Queue al Stack. Las Microtasks como las Promises tienen mayor prioridad y se procesan antes que las macrotasks como `setTimeout` al final de cada turno del Event Loop [19].

En la práctica esto significa que las promesas resueltas con `fetch` se ejecutan antes que un `setTimeout` pendiente del mismo ciclo. SGROAS aprovecha `async/await`, que es azúcar sintáctico sobre Promises, para escribir código asíncrono con apariencia síncrona dentro del módulo `api.js` [19].

2.4.2. Módulos ES6 vs CommonJS vs IIFE

Los módulos ES6 con `import` y `export`, estandarizados en ECMAScript 2015, ofrecen carga diferida, tree-shaking por parte de bundlers y ámbito de módulo que previene la contaminación del scope global. CommonJS con `require` y `module.exports` es el sistema de módulos de Node.js, es síncrono y no soporta tree-shaking nativo. Los IIFE fueron el patrón pre-módulos para aislar el scope pero resultan frágiles al escalar [19].

SGROAS usa el sistema de módulos ES6 nativo del navegador mediante `script type="module"`, lo que permite importar `api.js` y `ui.js` en `app.js` sin herramientas de compilación adicionales, manteniendo separación clara de responsabilidades.

2.4.3. Fetch nativo vs Axios

La API Fetch nativa basada en Promises es suficiente para la mayoría de casos. Sus limitaciones son que no lanza excepciones automáticas ante errores HTTP, hay que verificar `response.ok` manualmente, y requiere `AbortController` para cancelar solicitudes. Axios automatiza estas verificaciones, provee interceptores de solicitud y respuesta, y transforma automáticamente el JSON, pero añade aproximadamente 15KB al bundle. Para un proyecto sin dependencias de compilación, `fetch` con `async/await` y manejo explícito de errores mediante `try/catch` es la opción más adecuada [19].

2.4.4. Accesibilidad dinámica: focus trap y aria-live

Cuando se abre un modal dinámicamente es necesario transferir el foco al primer elemento interactivo dentro del diálogo y restringirlo para que no escape al contenido de fondo, técnica conocida como focus trap. SGROAS implementa esto en `ui.js`: al abrir el modal se ejecuta la función `trapFocus()` que escucha el evento `keydown` y redirige Tab y Shift+Tab al primer o último elemento focusable dentro del contenedor. Al cerrar, el foco retorna al elemento que lo tenía antes de abrir el modal [16].

Las regiones `aria-live` permiten que los lectores de pantalla anuncien cambios de contenido sin que el usuario haya puesto el foco en el elemento modificado. Se usa `aria-live="polite"` para anuncios no urgentes como resultados de búsqueda y `aria-live="assertive"` para mensajes críticos como errores de formulario. El atributo `aria-atomic="true"` indica que el lector debe anunciar toda la región modificada y no solo el fragmento cambiado [16].

3. Desarrollo

3.1. Descripción del proyecto

SGROAS es una aplicación web desarrollada para representar un Sistema de Gestión de Recursos Operativos, Administrativos y de Seguridad dirigido a cooperativas de transporte terrestre del Ecuador. Su finalidad es centralizar información básica de cooperativas, unidades, rutas, socios y estados operativos, mostrando los datos de forma organizada mediante tarjetas, formularios y paneles estadísticos.

La aplicación cuenta con varias secciones principales: inicio, directorio de cooperativas, control de flota, registro de cooperativas y estadísticas del sistema. Cada sección permite presentar información específica al usuario, facilitando la navegación dentro de una sola página mediante enlaces internos. El diseño se orienta a una experiencia clara, moderna y responsiva, permitiendo visualizar la información desde diferentes dispositivos.



Figura 1: Pantalla principal del sistema SGROAS

3.2. Entorno de desarrollo

El proyecto fue desarrollado en **IntelliJ IDEA**, utilizando una estructura de archivos organizada en carpetas para separar el código HTML, CSS y JavaScript. A diferencia de otros entornos que emplean extensiones como Live Server en Visual Studio Code, en este caso la aplicación fue ejecutada mediante el servidor local integrado de JetBrains, el cual genera una dirección local similar a:

```
http://localhost:63342/sgroas/index.html
```

El uso de este entorno permitió trabajar con recarga automática del navegador, organización del proyecto y edición directa de los archivos fuente. La aplicación fue probada en el navegador Google Chrome, desde donde también se realizaron validaciones con herramientas como Lighthouse y WAVE.

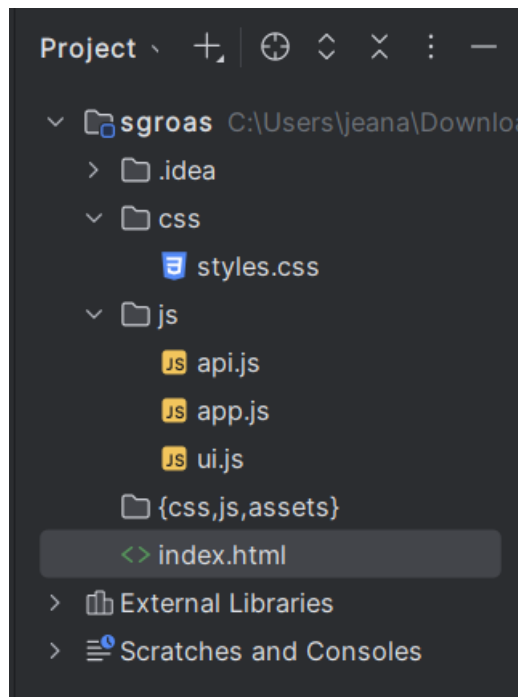


Figura 2: Estructura del proyecto SGROAS en IntelliJ IDEA

3.3. Estructura del proyecto

La aplicación se encuentra organizada en archivos separados para mejorar el mantenimiento del código. El archivo `index.html` contiene la estructura principal de la página; el archivo `styles.css` define el diseño visual, los colores, la responsividad y los componentes; mientras que los archivos JavaScript se dividen en módulos según su función.

```
sgroas/  
|  
+-- index.html  
+-- css/  
|   +-- styles.css  
+-- js/  
+-- api.js  
+-- app.js  
+-- ui.js
```

El archivo `api.js` se encarga de la comunicación con la API pública; `app.js` funciona como punto de entrada principal de la aplicación; y `ui.js` gestiona la interacción con el DOM, la búsqueda, el formulario, el modal y la renderización de tarjetas. Esta separación permite aplicar una estructura modular en JavaScript ES6+, haciendo que el código sea más ordenado y fácil de comprender.

3.4. HTML5 semántico aplicado

La estructura de SGROAS fue construida aplicando HTML5 semántico. Esto significa que se utilizaron etiquetas que describen el propósito de cada parte de la página, como `header` para la cabecera, `nav` para el menú de navegación, `main` para el contenido principal, `section` para dividir las áreas del sistema y `footer` para el pie de página. Este tipo de estructura permite

que el documento sea más comprensible para navegadores, motores de búsqueda y tecnologías de asistencia [1].

Además, el sistema incluye atributos de accesibilidad como `aria-label`, `aria-live`, `aria-current`, `role` y un enlace de salto al contenido principal mediante `skip-link`. Estos elementos ayudan a mejorar la navegación para usuarios que utilizan lectores de pantalla o navegación mediante teclado.

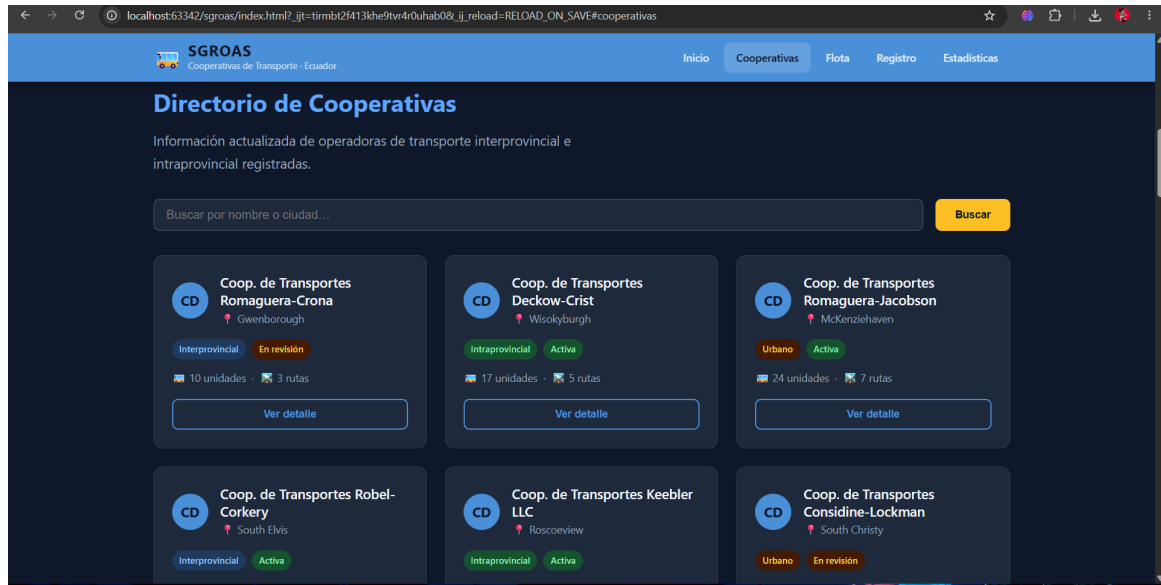


Figura 3: Directorio de cooperativas generado en la aplicación

3.5. CSS3 responsivo con Grid y Flexbox

El diseño visual de la aplicación fue implementado mediante CSS3, aplicando un enfoque *Mobile First*. Esto quiere decir que primero se definieron estilos pensados para pantallas pequeñas y luego se agregaron reglas mediante `media queries` para adaptar la interfaz a pantallas más grandes.

Para la distribución de elementos se utilizaron CSS Grid y Flexbox. CSS Grid permitió organizar secciones como el directorio de cooperativas, el panel de flota y las estadísticas en columnas adaptables. Por otro lado, Flexbox fue utilizado para alinear elementos internos como el encabezado, la barra de navegación, los botones y grupos del formulario. Estas técnicas permiten crear interfaces responsivas y ordenadas sin depender de estructuras rígidas [2], [3].



Figura 4: Panel de control de flota del sistema SGROAS

3.6. JavaScript ES6+ modular

La aplicación SGROAS utiliza JavaScript ES6+ para controlar la lógica principal del sistema, cargar datos externos, modificar dinámicamente el contenido de la página y gestionar la interacción del usuario. Para mantener el código organizado, se dividió la lógica en tres archivos principales: `api.js`, `app.js` y `ui.js`.

El archivo `api.js` contiene las funciones relacionadas con el consumo de la API pública y el cálculo de estadísticas. El archivo `app.js` funciona como punto de entrada de la aplicación, ya que inicializa los componentes de interfaz, carga los datos de cooperativas y controla posibles errores de conexión. Finalmente, el archivo `ui.js` se encarga de renderizar las tarjetas, mostrar estadísticas, activar el buscador, controlar el menú de navegación, abrir el modal de detalles, validar el formulario y guardar información en `localStorage`.

El uso de módulos permite separar responsabilidades y mejorar la mantenibilidad del proyecto. Además, se aplicaron características modernas de JavaScript como `import`, `export`, funciones flecha, `async/await`, manipulación del DOM y manejo de eventos. Estas prácticas son recomendadas en el desarrollo web moderno porque permiten crear aplicaciones más dinámicas, ordenadas y reutilizables [5], [6].

3.7. Consumo de API pública

Uno de los requisitos principales del proyecto fue consumir una API pública. Para cumplir con este punto, SGROAS utiliza la API de demostración JSONPlaceholder. Esta API proporciona datos de usuarios, los cuales fueron adaptados al contexto del sistema para representarlos como cooperativas de transporte.

Mediante la función `fetch()` se realiza una petición HTTP al recurso de usuarios de JSONPlaceholder. Luego, los datos recibidos son transformados en objetos con información relacionada con cooperativas, como nombre, ciudad, correo, teléfono, unidades, rutas, tipo de servicio, estado y número de socios. Esta adaptación permitió simular un directorio funcional de cooperativas, manteniendo el enfoque del sistema dentro del transporte terrestre ecuatoriano.

El uso de `async/await` permitió manejar la carga de datos de forma clara y ordenada. Además, se implementó control de errores para mostrar mensajes cuando existe un problema de conexión o cuando los datos no pueden ser cargados correctamente. Este proceso demuestra la integración entre una fuente de datos externa y la interfaz web de la aplicación [4], [8].

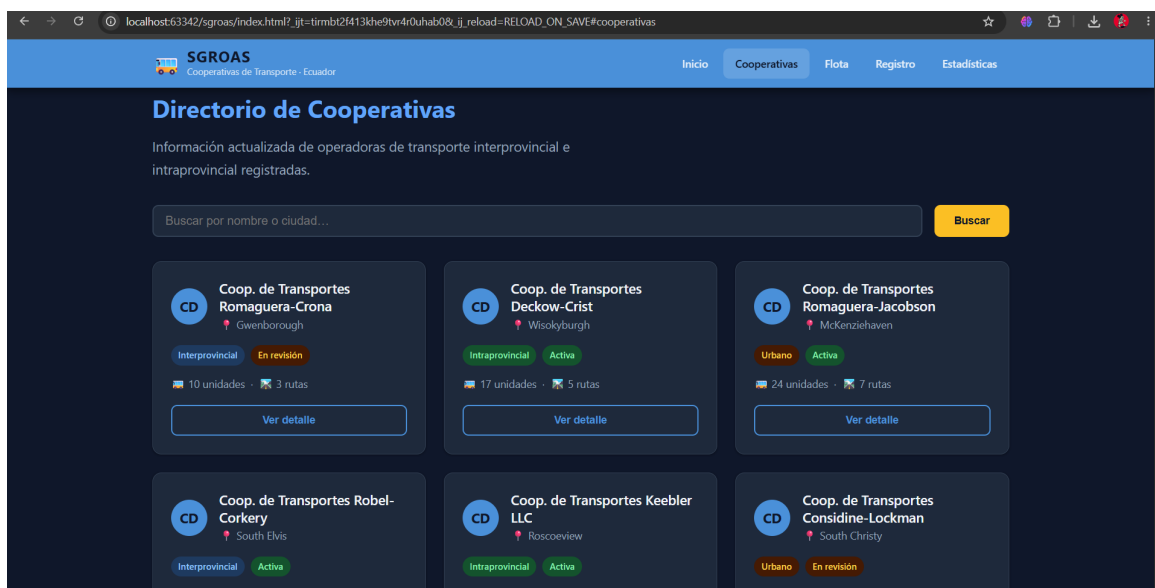


Figura 5: Datos de cooperativas cargados desde la API pública

3.8. Funcionalidades del sistema

La aplicación SGROAS presenta diferentes funcionalidades orientadas a la gestión visual de cooperativas de transporte. En la pantalla de inicio se muestra una descripción general del sistema y estadísticas principales como el número de cooperativas, unidades activas y rutas registradas.

En la sección de cooperativas se presenta un directorio mediante tarjetas dinámicas. Cada tarjeta muestra el nombre de la cooperativa, ubicación, tipo de servicio, estado, número de unidades y rutas. También se incluye un botón para visualizar más detalles mediante una ventana modal.

La sección de flota permite observar información resumida sobre las unidades registradas, rutas activas, socios registrados y cooperativas activas. Esta información se calcula automáticamente a partir de los datos obtenidos desde la API.



Figura 6: Resumen operativo de la flota registrada

La sección de registro contiene un formulario para ingresar información de una nueva cooperativa. Este formulario utiliza diferentes tipos de campos HTML, como texto, correo electrónico, teléfono, número, fecha, rango, contraseña, archivo, lista desplegable y casilla de verificación. También incluye validaciones para evitar que se ingresen datos incorrectos, como un RUC incompleto o un correo electrónico no válido.

The screenshot shows the 'Registro de Cooperativa' (Cooperative Registration) form in the SGROAS application. The form is titled 'Registro de Cooperativa' and includes the instruction: 'Complete el formulario para registrar una nueva cooperativa en el sistema SGROAS.' The form is divided into a section titled 'Datos institucionales' with the following fields:

- Nombre de la cooperativa ***: Ej. Cooperativa de Transportes Flota Imbabura
- RUC de la cooperativa ***: 0000000000001. Below the field, it says: 'Ingrese los 13 dígitos del RUC sin guiones.'
- Correo electrónico ***: contacto@cooperativa.ec
- Teléfono de contacto ***: +593 99 123 4567

Figura 7: Formulario de registro con validaciones

Finalmente, la sección de estadísticas muestra un resumen general del sistema. Allí se presentan datos como total de cooperativas, unidades registradas, rutas en operación, socios activos, cooperativas interprovinciales e intraprovinciales. Esta información permite visualizar el estado general de la red de transporte representada en la aplicación.

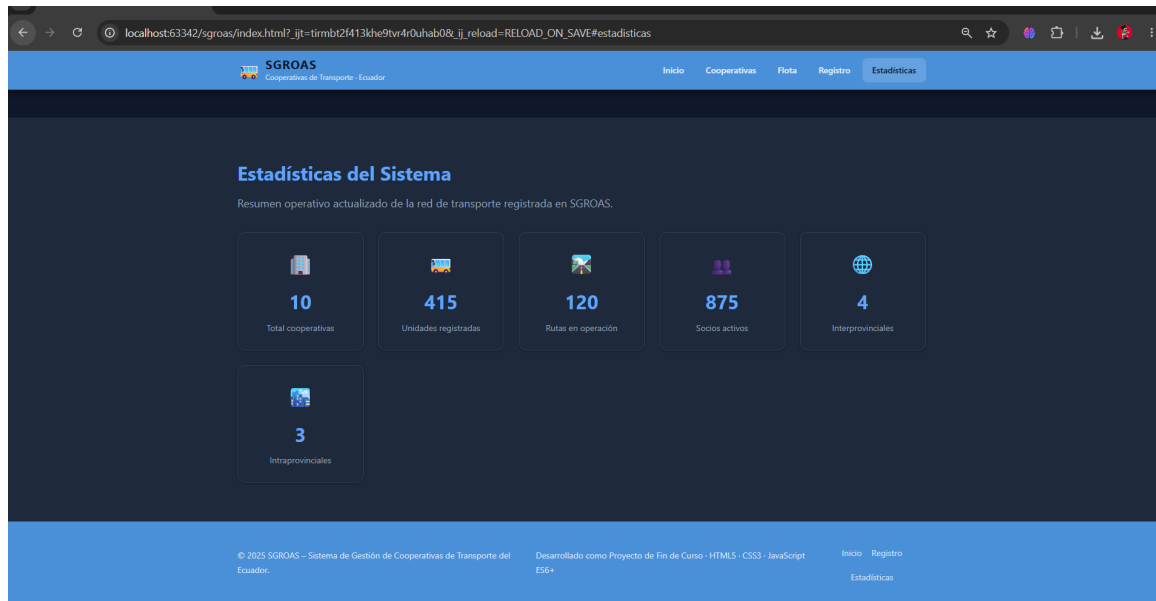


Figura 8: Panel de estadísticas del sistema SGROAS

3.9. Validaciones y accesibilidad

El sistema incorpora validaciones en el formulario de registro para mejorar la calidad de los datos ingresados por el usuario. Por ejemplo, el campo RUC exige exactamente 13 dígitos, el correo electrónico debe cumplir con el formato correcto, el número de unidades debe estar dentro de un rango permitido y la contraseña debe tener una longitud mínima con requisitos básicos de seguridad.

También se aplicaron elementos de accesibilidad en la estructura HTML. Entre ellos se incluyen etiquetas semánticas, textos alternativos o descriptivos mediante atributos ARIA, navegación por teclado, enlace para saltar al contenido principal, estados de alerta accesibles y control de foco en el modal. Estas prácticas ayudan a que la aplicación pueda ser utilizada por una mayor variedad de usuarios y dispositivos [7], [10].

3.10. Validación con Lighthouse y WAVE

Para comprobar la calidad técnica del sistema SGROAS, se realizaron pruebas con Lighthouse y WAVE. Lighthouse permitió evaluar métricas relacionadas con rendimiento, accesibilidad, buenas prácticas y SEO. La primera evaluación obtuvo un resultado de 96 en rendimiento, 97 en accesibilidad, 100 en buenas prácticas y 100 en SEO. Después de realizar ajustes, el rendimiento aumentó a 99, la accesibilidad se mantuvo en 97, las buenas prácticas quedaron en 96 y el SEO se mantuvo en 100.

Métrica	Antes	Después
Performance	96	99
Accessibility	97	97
Best Practices	100	96
SEO	100	100

Cuadro 5: Resultados obtenidos en Lighthouse

Los resultados muestran una mejora en el rendimiento general de la aplicación. La accesibilidad se mantuvo en un valor alto, mientras que SEO conservó la puntuación máxima. Aunque la métrica de buenas prácticas disminuyó de 100 a 96, el resultado sigue siendo favorable para una aplicación académica desarrollada en entorno local.



Figura 9: Evaluación Lighthouse antes de los ajustes



Figura 10: Evaluación Lighthouse después de los ajustes

Además, se utilizó la herramienta WAVE para revisar aspectos de accesibilidad visual y estructural. En esta evaluación se detectaron 0 errores generales, 0 alertas, 17 características, 14 elementos de estructura y 106 elementos ARIA. Sin embargo, también se identificaron 19 errores de contraste, por lo que se reconoce como mejora pendiente ajustar ciertos colores entre texto y fondo para facilitar la lectura.

Elemento evaluado	Resultado
Errores generales	0
Errores de contraste	19
Alertas	0
Features	17
Estructura	14
Elementos ARIA	106

Cuadro 6: Resultados obtenidos en WAVE



Figura 11: Evaluación de accesibilidad realizada con WAVE

3.11. Validación W3C HTML y CSS

Además de las pruebas realizadas con Lighthouse y WAVE, se utilizaron los validadores oficiales del W3C para revisar la estructura del código HTML y la sintaxis de la hoja de estilos CSS. Estas herramientas permiten comprobar si el documento cumple con estándares web y si existen errores que puedan afectar la compatibilidad del sitio en distintos navegadores [11], [12].

En la validación HTML se comprobó el archivo principal `index.html`. El resultado mostró advertencias relacionadas principalmente con el uso de roles ARIA innecesarios en elementos semánticos como `header`, `nav`, `ul` y `footer`. Estas advertencias no representan errores críticos de funcionamiento, sino recomendaciones de mejora, debido a que HTML5 ya proporciona significado semántico a dichas etiquetas de forma nativa.

Elemento evaluado	Resultado
Errores críticos HTML	0
Advertencias HTML	13
Mensajes informativos	17

Cuadro 7: Resultados obtenidos en W3C HTML Validator

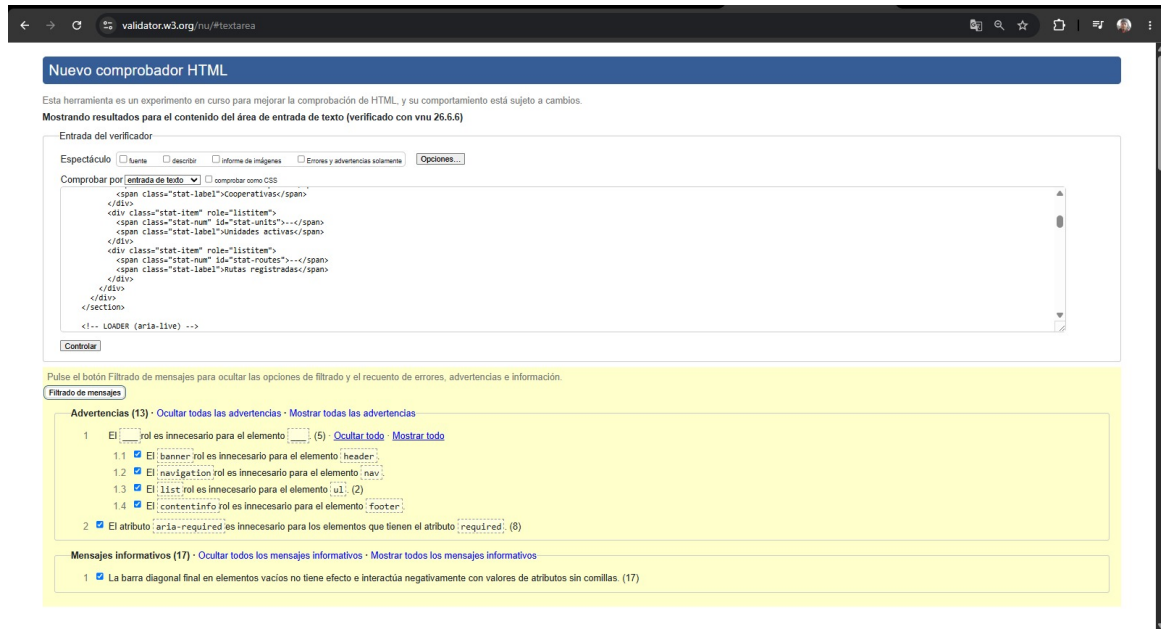


Figura 12: Validación del archivo HTML mediante W3C Validator

En la validación CSS se revisó el archivo `styles.css`. El resultado fue favorable, ya que el validador indicó que no se encontraron errores y que la hoja de estilos es válida para CSS versión 3 + SVG. Esto confirma que la estructura de estilos utilizada en el proyecto cumple correctamente con la sintaxis esperada por el estándar CSS.

Elemento evaluado	Resultado
Errores CSS	0
Estado de validación	CSS válido
Versión evaluada	CSS 3 + SVG

Cuadro 8: Resultados obtenidos en W3C CSS Validator



Figura 13: Validación del archivo CSS mediante W3C CSS Validator

Con estas validaciones se evidencia que la aplicación SGROAS fue revisada no solo desde el rendimiento y la accesibilidad, sino también desde el cumplimiento de estándares HTML y CSS. Aunque el HTML presentó advertencias menores, estas pueden corregirse posteriormente eliminando roles redundantes en etiquetas que ya son semánticas por defecto.

4. Conclusiones

El desarrollo de la aplicación web SGROAS permitió aplicar de forma práctica los conceptos de HTML5 semántico, CSS3 responsivo y JavaScript ES6+. La estructura del sistema fue organizada mediante secciones claras como inicio, cooperativas, flota, registro y estadísticas, lo que facilitó la navegación del usuario dentro de una sola página. Además, el uso de etiquetas semánticas y atributos de accesibilidad permitió mejorar la comprensión del contenido y la interacción con la interfaz.

El diseño responsivo implementado con CSS Grid, Flexbox y media queries permitió que la aplicación se adapte a diferentes tamaños de pantalla. Esto es importante porque una aplicación web moderna debe ser funcional tanto en computadoras como en dispositivos móviles. También se aplicaron variables CSS, modo oscuro nativo, tarjetas informativas y una distribución visual ordenada, logrando una interfaz más atractiva y fácil de utilizar.

La implementación de JavaScript ES6+ permitió crear una aplicación dinámica mediante módulos separados. El archivo `api.js` se encargó del consumo de la API pública JSONPlaceholder, `app.js` coordinó la carga de datos y `ui.js` gestionó la interacción con la interfaz. Gracias a esta organización, el sistema pudo mostrar cooperativas, calcular estadísticas, realizar búsquedas, validar formularios, abrir ventanas modales y almacenar registros en `localStorage`.

Finalmente, las pruebas realizadas con Lighthouse y WAVE permitieron comprobar la calidad técnica de la aplicación. En Lighthouse se observó una mejora en rendimiento, pasando de 96 a 99, manteniendo una accesibilidad alta y un SEO de 100. En WAVE no se detectaron errores generales ni alertas, aunque sí se identificaron errores de contraste que deben corregirse en futuras versiones. Por lo tanto, se concluye que SGROAS cumple con los requisitos principales del proyecto, pero aún puede mejorar en aspectos visuales de accesibilidad.

5. Repositorio en GitHub

Como parte de la entrega del proyecto, el código fuente de la aplicación web SGROAS fue subido a un repositorio en GitHub. Esto permite evidenciar la organización del trabajo, conservar una copia del proyecto y facilitar la revisión del código por parte del docente.

El repositorio contiene los archivos principales de la aplicación, incluyendo el documento `index.html`, la hoja de estilos `styles.css` y los módulos JavaScript `api.js`, `app.js` y `ui.js`. De esta manera, se demuestra que el proyecto fue estructurado correctamente y respaldado mediante una plataforma de control de versiones.

Enlace del repositorio:

<https://github.com/Alxjandr07/GRUPO-E-PRACTICA-EXPERIMENTAL-UNIDAD-1.git>

6. Referencias

- [1] Mozilla Developer Network, “Semantic HTML,” MDN Web Docs. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/curriculum/core/semantic-html/>. [Consultado: 10-jun-2026].
- [2] Mozilla Developer Network, “CSS grid layout,” MDN Web Docs. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Grid_layout. [Consultado: 10-jun-2026].
- [3] Mozilla Developer Network, “Flexbox,” MDN Web Docs. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Flexbox. [Consultado: 10-jun-2026].
- [4] Mozilla Developer Network, “Using the Fetch API,” MDN Web Docs. [En línea]. Disponible en: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch. [Consultado: 10-jun-2026].
- [5] Mozilla Developer Network, “JavaScript,” MDN Web Docs. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. [Consultado: 10-jun-2026].
- [6] Mozilla Developer Network, “JavaScript modules,” MDN Web Docs. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>. [Consultado: 10-jun-2026].
- [7] Mozilla Developer Network, “ARIA,” MDN Web Docs. [En línea]. Disponible en: <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>. [Consultado: 10-jun-2026].
- [8] Typicode, “JSONPlaceholder Guide.” [En línea]. Disponible en: <https://jsonplaceholder.typicode.com/guide/>. [Consultado: 10-jun-2026].
- [9] Chrome Developers, “Introduction to Lighthouse.” [En línea]. Disponible en: <https://developer.chrome.com/docs/lighthouse/overview>. [Consultado: 10-jun-2026].
- [10] WebAIM, “WAVE Web Accessibility Evaluation Tools.” [En línea]. Disponible en: <https://wave.webaim.org/>. [Consultado: 10-jun-2026].

- [11] W3C, “Markup Validation Service.” [En línea]. Disponible en: <https://validator.w3.org/>. [Consultado: 10-jun-2026].
- [12] W3C, “CSS Validation Service.” [En línea]. Disponible en: <https://jigsaw.w3.org/css-validator/>. [Consultado: 10-jun-2026].
- [13] GitHub, “Repositorio del proyecto SGROAS.” [En línea]. Disponible en: <https://github.com/Alxjandr07/GRUPO-E-PRACTICA-EXPERIMENTAL-UNIDAD-1.git>. [Consultado: 10-jun-2026].
- [14] T. Berners-Lee, R. Fielding y H. Frystyk, “Hypertext Transfer Protocol – HTTP/1.0,” RFC 1945, Internet Engineering Task Force, May 1996. [En línea]. Disponible en: <https://www.rfc-editor.org/rfc/rfc1945>. [Consultado: 10-jun-2026].
- [15] A. Russell, “Progressive Web Apps: Escaping Tabs Without Losing Our Soul,” 2015. [En línea]. Disponible en: <https://infrequently.org/2015/06/progressive-apps-escaping-tabs-without-losing-our-soul/>. [Consultado: 10-jun-2026].
- [16] W3C Web Accessibility Initiative, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, Jun. 2018. [En línea]. Disponible en: <https://www.w3.org/TR/WCAG21/>. [Consultado: 10-jun-2026].
- [17] WHATWG, “HTML Living Standard,” 2024. [En línea]. Disponible en: <https://html.spec.whatwg.org/multipage/>. [Consultado: 10-jun-2026].
- [18] R. Eckles et al., “CSS Grid Layout Module Level 1,” W3C Candidate Recommendation, Oct. 2017. [En línea]. Disponible en: <https://www.w3.org/TR/css-grid-1/>. [Consultado: 10-jun-2026].
- [19] TC39, “ECMAScript 2015 Language Specification ES6,” ECMA International, Jun. 2015. [En línea]. Disponible en: <https://262.ecma-international.org/6.0/>. [Consultado: 10-jun-2026].
- [20] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994. ISBN: 978-0-201-63361-0.